

AWS CLOUD PRACTITIONNER

- in the cloud we are having many services that work to achieve many principles, among them we have:
 - **Design for failure:** where we make sure components do not get failed
 - **Decouple components:** where we make sure that if one component is down in our architecture, the other components do not get affected by that.
 - **Implement elasticity:** where you scale up and down based on the load.
 - **Think parallel:** where we increase speed, performance by implementing parallel wherever possible.

AWS Well Architecture Framework

- **PROCESS**
- Performance efficiency ==> speed and right resources usage, e.g. using the right type of ec2
- Reliability ==> recover from failures and HA
- Operational excellence ==> making sure everything is working smoothly and the same way and everything is up to date (monitoring , running patches , continuous improvements)
- Cost optimization
- E EMPTYYY
- Security
- Sustainability ==> minimizing the footprint on AWS by deleting unused unnecessary things

AWS Cloud Adoption Framework

1. **Business**
2. **People**
3. **Governance**
4. **Platform**
5. **Security**
6. ****Operations**

- ACRONYM:
 - BP-GPSO
- How to Remember Them:
 - The AWS CAF perspectives are divided into two main categories:
 - **Business Capabilities (Focus on Strategy & Culture):**

- **Business:** Aligns cloud investments with digital transformation goals.
- **People:** Focuses on culture, leadership, and workforce fluency/training.
- **Governance:** Manages risks, project benefits, and cloud financial management.
- **Technical Capabilities (Focus on Execution & Operations):**
 - **Platform:** Guides building scalable cloud infrastructure and modern app architectures.
 - **Security:** Ensures confidentiality, data protection, and compliance.
 - **Operations:** Focuses on observability, patch management, and running live workloads reliably.

AWS Shared Responsibility Model

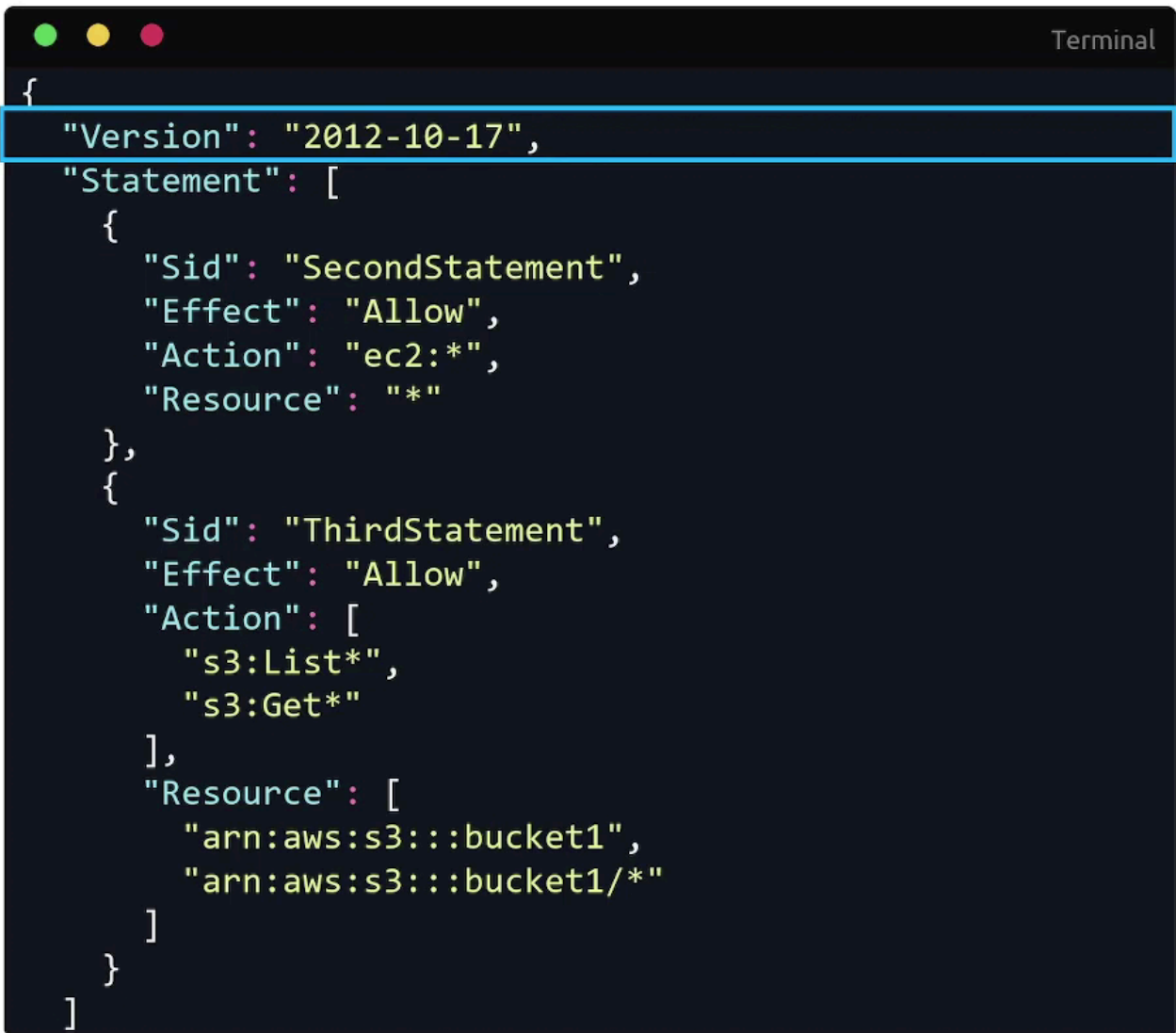
- it is to say that AWS is not responsible for everything , it is responsible for the security OF the cloud and the client is responsible of the security IN the cloud
- so any infrastructure of the cloud, AWS is responsible of that
- but anything else that the client puts in the cloud: APPs, DATA, FILES, ENCRYPTION, USERS in their org.. , HE IS RESPONSIBLE FOR THEM

Security and Compliance

- Compliance and regulatory frameworks are a set of rules and guidelines (national or international) for the orgs to follow and adhere to.
- AWS Compliance and adhering can be found in the service AWS Artifact
- AWS Artifact is a service that has all the details about AWS compliance
 - it proves that AWS is compliant
- AWS Compliance Center contains all the guidelines related to the Cloud industry and how many orgs reacted to them depending on your country etc.
- **AWS Audit Manager:** is a AWS service that helps you to know if your deployed resources on AWS comply or adhere to specific frameworks
 - it answer to the question : is my resources comply to HIPAA (or some framework)?
 - it also gives you reports to show them to some third parties that want to know if your resources comply to some framework
- **AWS Config** service checks if your resources comply to some defined rule you define inside AWS (e.g. all s3 buckets must be private) ==> it is called Config Rule
 - then AWS will run that rule on your resources and it will give you a compliance report about each rule

IAM

- IAM is a service in AWS that handles users/apps permissions and restrictions
- IAM contains all the users authenticated to your AWS account with their exact permissions
- when firstly creating an AWS account a **ROOT USER** will be created that has permissions to every single thing in your AWS account
 - but it is not recommended to work with the root user ==> it is recommended that you create a new user in IAM and give them the permission you need and you start using that user (usually ADMINISTRATOR), because if you keep using the ROOT USER and they somehow the credentials get compromised you are fked up
- you can add as many users as you want in IAM
- when a user is firstly added to IAM they have 0 permissions
- you must attach them permissions using **Policies**
- **Policies** define what user can do what in AWS
- here an example of a policy file:



```

{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "SecondStatement",
      "Effect": "Allow",
      "Action": "ec2:*",
      "Resource": "*"
    },
    {
      "Sid": "ThirdStatement",
      "Effect": "Allow",
      "Action": [
        "s3:List*",
        "s3:Get*"
      ],
      "Resource": [
        "arn:aws:s3:::bucket1",
        "arn:aws:s3:::bucket1/*"
      ]
    }
  ]
}

```

- **Policies** are separate from users, means you can create a single policy and attach it to many users

- AWS has many managed policies ==> at the same time you can create your own policies using the file describe above with defining the appropriate rules that you want / or by using the UI.
- **GROUPS** are used to collect many users in a single group and attach them all the same policies ==> means when you attach a group some policy ==> all the users in that group will inherit that policy
- e.g : HR Group, FINANCE Group, DEV Group
- **ROLES** are created to assign a user some role with the corresponding permissions TEMPORALY
- you can create Developer Role and attach it to someone > means they will inherit all the policies for that ROLE ==> then when finish the mission you remove that ROLE from them as simple as that = > better then creating the policies and attaching then removing them everytime, that will be tedious
- **LEAST PRIVILEGE PERMISSION** ==> AWS recommends to grant users the **bare minimum** permission they need ==> e.g a user has to only view S3 object ==> instead of giving them a policy to access all S3 CRUD ==> give them a very specific policy ==> allow them to only **VIEW S3**

Organizations

- **organizations** help you manage many AWS account instead of managing them separately
- an **organization** can have many AWS accounts
- **Organizational Units** (OUs) acts like Group in IAM, it helps attach rules etc to similar accounts
- **Service Control Policy** (SCP) restrict what an account can do
- **Service Control Policies** (SCPs) can be attached to an account or to an organizational unit

Security Services

Prevention

- **Web Application Firewall**: it receives http/s requests for some defined AWS resources, analyze them (IP , country , request size, string & regex matching , sql & xss), then it decides what to do (ALLOW BLOCK CAPATSHA)
- **AWS Shield**: prevents apps from DDOS attacks
- **AWS Network Firewalls**: it defines rules on the traffic entering and leaving the VPC

Detection

- **AWS Inspector:** it works with EC2, EC2 images (AMI), Lambda Functions , and it analyzes the OSs images , the packages that the workload running on those EC2 works with , packages and dependencies that code running on Lambda functions depends on
- it will give you a report about any bugs found in any package or OS image
- it will run automatically whenever needed, e.g. a new package is introduced
- **AWS GuardDuty:** it will analyze all the activities logs on AWS infrastructure, it uses ML to identify any suspicious API activities or logs , then report them
- **Amazon Detective:** it takes logs and findings from other services, then create advanced visualizations that are easy to navigate , using ML and advanced statistics
- **AWS Config:** it logs all the configurations made on your AWS resources, any changes and configs on resources will be logged
- **CloudTrail:** it logs users' activities in AWS, any action a user does, it is logged down
 - it answers the question: **WHO did WHAT in an AWS account and WHEN**
- **Security Lake:** it centralizes logs and security data from various sources (on prem, third parties ect) , normalizes them into OCSF format so that other customers can process it easily
 - stores them in s3
 - OCSF is an normalized format for cybersecurity data to easily process it, instead of nailing things differently, people just agree on one language to use in cybersecurity logs and data , to eliminate parsing scripts ect, so that customers find everything ready
- **AWS Macies:** scans s3 buckets for any sensitive data then notifies u
- **AWS Security Hub:** it gathers together all other security services in a single dashboard,
 - provides security monitoring and governance across multiple AWS accounts
 - it aggregates all logs from other accounts, services into one view and tells you what each service has found
 - it checks compliance against common standards
 - gives compliance score in a beautiful UI

Management

- **Firewall Manager:** it is a central location for managing different firewall rules to share them later among the different SECURITY SERVICES around different accounts, apps, orgs
 - it eliminates the need of managing the firewall rules of many accounts separately
- **Resources Access Manager:** it helps to share resource among many AWS accounts easily, you might want an EC2 instance from account A talks to another EC2 instance from account B ==> so you would attach the same subnet for account A & B then attach those EC2 instances to that subnet
- **AWS Cognito:** it gives you an Authentication as a service with sign up / log in , ready to implement with your mobile / web app

- Cognito authenticated users can have their own policies or IAM rules to access the aws account resources
- it aggregates all the oauth providers (google , facebook , apple ...), and instead of writing custom code for each provider, you juust use Cognito and route it to the specific provider
- **IAM:** it defines who can access what
- **IAM Identity Center:** it is IAM for many AWS accounts
- **Secrets Manager:** instead of storing secrets in .env ect, you simply store them on that service , then your app will access them
- **AWS Certificate Manager:** it manages SSL/TLS certifications that are used by AWS services, like ALB , you can simply attach an ACM certificate to the ALB and it will work freely and you are happy with the https to your Load Balancer, then your LB will simply use plain http to communicate with your deployed app (on EC2 instances or whatever)
 - it is not downloadable to use it inside EC2 with nginx or idk
- **AWS Private Certificate Authority:** it generates to you downloadable certificate, but they are not trusted globally, so your users will allways get the ANNOYING warning
 - they are not recommended to use with deployed web servers globally
 - you use them internally with your EC2 instances for exemple
 - we can use them in ZERO TRUST approaches, where we dont trust the sentence: " we are inside VPC it is okay to not encrypt"
- **Key Management Service:** it manages all the keys u use to encrypt/decrypt data in AWS,
 - might be used in s3, dynamoDB
- **CloudHSM:** an advanced version of KMS, in KMS AWS handles all the hardware where the keys are stored and how etc. , in CloudHSM you have a dedicated hardware device where you are responsible for storing the keys your self ect

AWS Infrastructure

- **Regions** are locations around the world that AWS dedicated to build their data centers in, one region can contain many datacenters
 - each region can have different services, with different prices etc.
 - so make sure you read about that region and if it convers all your needs before deploying there
- **Availability zones** are under regions, each regions can have many AZ, an AZ is just a datacenter within a region
 - AZs within a single region are isolate and independent, if one fails others wont get affected
 - so it is recommended to duplicate workloads in more than one AZ
- **Edge locations** are locations around the world used at huge scale to reduce latency,

- they work within AWS CDN,
- they often just deliver cash things like static websites, videos, images
- CloudFront
- **Local Zones** are extensions of AWS regions , they are not inside the region but they are close to it, unlike Edge Locations where they are scattered randomly
 - LZ provides real compute power etc and they actually run real services
 - their first purpose is ultra low latency
 - they are used in video games ect where the business want their gamers to join the very close server, EL wont be able to achieve real computing

Networking in AWS

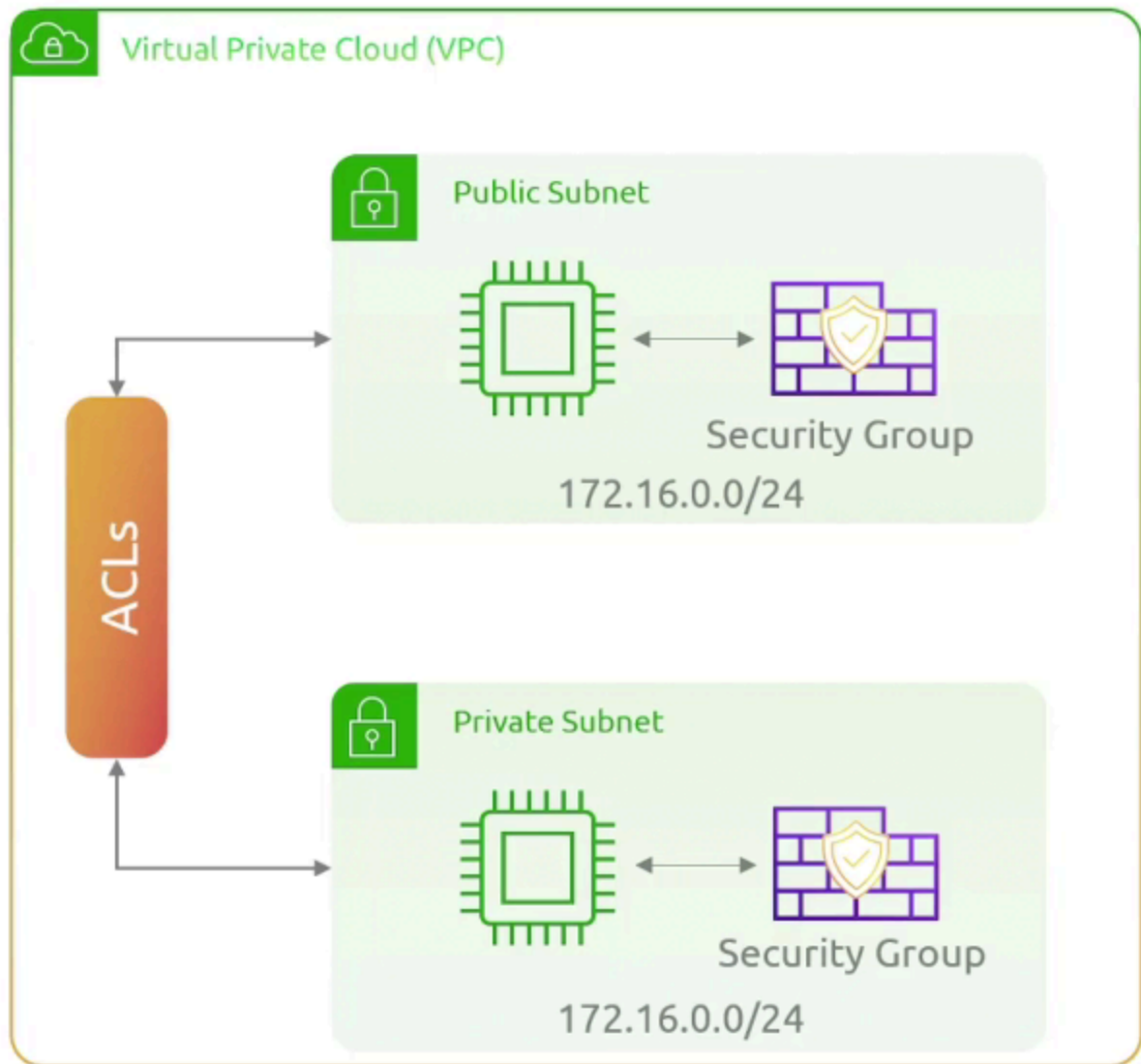
- each AWS account has one or more Virtual Private Cloud VPC
- VPC separates your own resources from other's resources in the cloud, it makes your resources private only to you
 - VPCs are per region , you cannot have a VPC that includes two separate regions
 - VPCs has block of IP @ called CIDR block
 - VPCs can have many subnets
- Subnet is a range of IP@ within the CIDR
 - Subnets reside within a single AZ
 - Subnets can be private or public
 - Subnets when created firstly are by default private
 - if you want to make a subnet public you use:
 - Internet Gateway: you create IGW , attach the outgoing traffic in the routing table of that subnet to point to that IGW
 - NAT Gateway: this is used in private subnets only , where resources within here initiate the connection to the internet , you create a NAT GW and point all outgoing traffic to that NAT GW in the routing table
 - the first 4 addresses & the last address from the subnet cannot be used
 - the first one: x.x.x.0 is the network address
 - x.x.x.1 to x.x.x.3 are AWS reserved
 - x.x.x.255 is the broadcast @
- to access your private cloud resources from your on prem you use:
 - **Virtual Private Gateway**: enables secure external access to private cloud resources, from your on prem to cloud for eg
 - **Direct Connect** (DX): high speed & low latency

Default VPC

- when you firstly create your aws account you will have:
 - default VPC in each existing region
 - each VPC has a CIDR block of /16 (256 * 256 addresses)
 - each default VPC will have a default Internet Gateway
 - those VPCs each has a MAIN ROUTING TABLE that is public, has outgoing traffic pointing to the default created IGW
 - those default VPCs have default Subnets (/20 subnets)
 - each default subnet is public (assigned to the MAIN ROUTING TABLE)
 - each AZ will have a default /20 subnet
 - those default subnets are accessible from the internet and are public by default (but those that you create by yourself will be private by default)
 - means each resource (EC2 instance ..) you create will be public if you maintain the default parameters
 - there will be security groups set by default to OUTBOUND for every resource within that subnet
 - default NACL set to INBOUND and OUTBOUND
 - Security groups work at the resource level
 - NACL work at the whole subnet network level

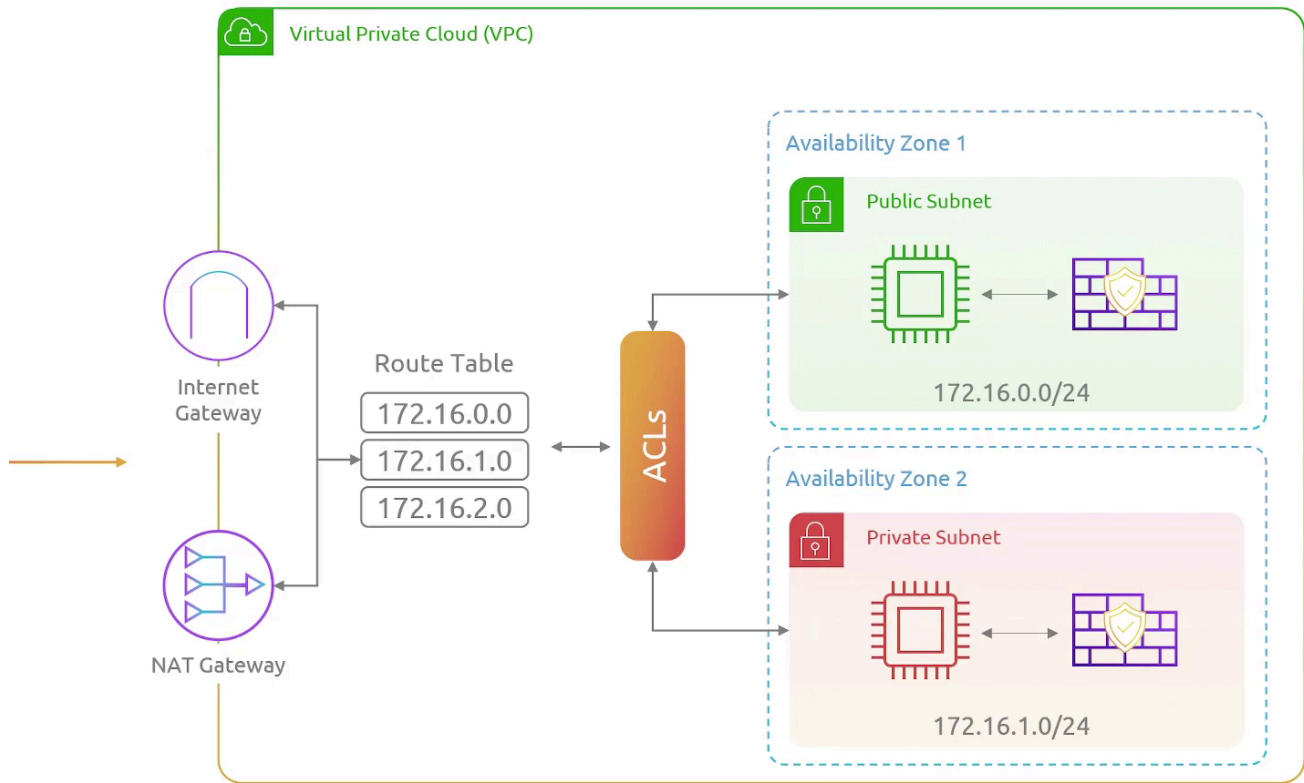
Firewalls

- there are two types of firewalls
 - STATELESS: inbound and outbound are independent, you have to set them both explicitly,
 - so for a request on 443 on some resource, you have to add ALLOW 443 in the INBOUND
 - and for the response , you have to add it in the OUTBOUND also with that **ephemeral** port (but actually you don't know that ephemeral port , so you will ALLOW the range of ephemeral PORTS , that range depends on the OS that assigns to each request's response)
 - STATEFULL: you only care about the rule to allow the request to PASS / LEAVE , the response of that request will be automatically ALLOWED
- NACL are STATELESS firewalls they care about traffic leaving and entering the subnet
- Security Groups are firewalls for individual resources , they are STATEFULL firewalls



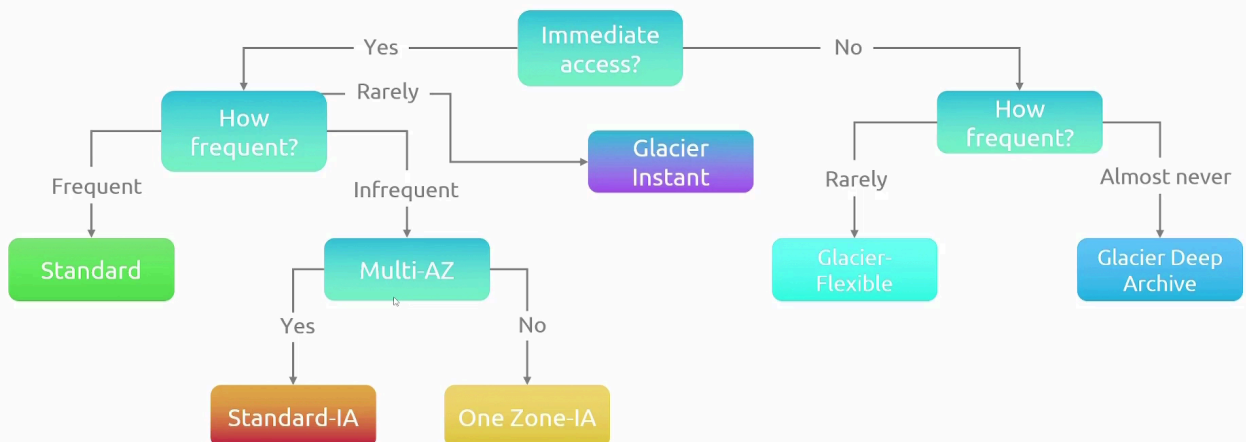
.

Overview Netowrk Schema



Storage

- AWS supports many types of storage
 - Block storage
 - File storage
 - Object storage
- for object storage, s3 is the service responsible for that
- s3 has many offers and options , it all depends on how frequently you access the data, and how quickly you want the data to be retrieved



Types of storage

- we have three types of storage
- Block Storage
 - this is one per instance
 - it is the hard drive of the machine
 - AWS supports it using two services
 - **Instance blocks**: common for HPCs, not persisting data, it is fast but not persistent
 - **Elastic Block Store**: the most common , it is persistent , and it is independent of the instance, if you detach it from one instance, you can use it with another
- File Storage
 - it can be shared by many instances at the same time, and they all can see real time updates
 - AWS supports this using the service:
 - **Elastic File System EFS for linux** instances because it uses a protocol that is supported only by Linux
 - **FSx for windows** because it uses a protocol that is supported by WINDOWS
- Object Storage
 - the very high level storage
 - it stores normal data like media, websites ect
 - AWS supports it using the service:
 - Simple Storage Service or S3

AWS Architecture for EBS

- EBS are one to one to instances
- an EBS and an instance must be on the same AZ
- EBS are not duplicated around many AZ
- they are duplicated around many drives in that AZ

AWS Architecture for EFS

- EFS are many to many , many instances can mount the same EFS, and an instance can have many EFS
- EFS and the instance can be in separate AZ
- are EFS duplicate ? it depends on the option you are using (see diagram above)
 - standard option it duplicates them around AZ

Compute

- AWS offers many services for compute resources
- one of them is EC2
 - it simply offers VMs
 - those VMs are so customizable, you can put the configuration that you want
 - they use images from **Amazon Machine Images**, which is a service that has many OS images
- we also have Lambda Functions
 - it is a serverless service for compute
 - you can run your code without thinking of infrastructure, since it is all managed by AWS
 - you can add triggers when your functions should be running , e.g. an object added to s3
- for container orchestration there is two main service in AWS
- ECS and EKS
- Elastic Container Service
 - is a fully AWS managed container orchestration service, it manages everything in the cluster , from control plane to workload plane
- Elastic Kubernetes Service
 - is a partly AWS managed container orchestration service
 - it is based on KUBERNETES
 - it only manages the control plane
 - the workload plane it is up to you to handle and to manage

Databases

- AWS offers managed databases for both relational and non relational DB
- Amazon Aurora:
 - is the AWS managed service for relational databases
 - it leverages the **low cost of PostgreSQL** with the **enterprise performance of Microsoft** and oracle db engines
 - auto HA , it replicates 6 copies across 3 AZ
- Amazon DynamoDB:
 - is the AWS managed service for the nonrelational databases
 - it is serverless ==> you dont care about infra
 - it is designed for high performance internet scale applications
 - it can handle 10 trillion requests per day
 - it offers limitless availability 99.999%

Billing

Types of billing

- there two types of billing in AWS, it all depends on the service
 - Duration based billing
 - Request based billing
- Duration based billing you pay for the time you allocate the resource
 - they dont care if the resource is handling users or is idle, as long as it is running , you will pay for it
 - if resource is "running" or "available" you get billed regardless of the traffic
 - the billing is per time (hours etc)
 - e.g. ec2 , managed compute services like eks ect
- Request based billing you pay only for the time your users hit the service
 - so if there is no one using your resource you will pay nothing
 - but once the user hits your resource you are billed for that
 - if traffic is 0 you get 0\$ billing
 - the billing is per requests
- NB: it is not optional for you that you choose which type of billing when you prvision the resource,
 - it is hardcoded that some service are only DURATION BASED BILLING, others are REQUEST BASED, and others are HYBRID

Options of billing

- we have four options to set your bill
 - **on demand**
 - **spot**
 - **reservation**
 - **saving plans**
- **ON DEMAND** is that you pay as you go, there is no contract on the resources, you just pay for what you are using
 - on demand can be used with both types of billing, **duration based** or **request based**
 - for duration based services you will be billed per time about the resource
 - for request based services you will simply be billed per request on the lambda function or wutever
- **SPOT** means you take some excess resource of AWS and use them at lower price, until a paying client that is ready to pay for that EC2 to take it from you,

- where you will have 2mins warning before your local data inside the ec2 for eg are lost from that spot instance, data in ebs ect survive
- we can use this option only with **duration based** billing services , where the infra is all yours
- in request based services, the services are aws managed, the underlying infra (os etc) is aws managed so u cannot take API GATEWAY function as yours lol
- **SAVING PLANS** here you sign a contract with AWS telling them you will be paying x\$ per hour for x years , they will give you huge discounts for that x\$ per hour
 - used only with **duration based** services since the metric of saving plans is time
 - NB: once you exceed the committed amount , you will start to be billed **ON DEMAND**
 - NB: **Compute Savings Plans** can be used on AWS Lambda which is a request based service, because AWS Lambda is billed partly on the execution time along side with request hitting it ,
- **RESERVATION** where you reserve some instance only for you , you reserve the infrastructure for you, it can be ec2 , database, storage ...
 - used only with **duration based** billing because aws wont track the request hit on ec2 lol
 - and you will be charged for that food regardless whether you are getting it or not lol

EC2 instances billing

- the billing is depending on the OS
 - Linux ect are billed per second with 1min as minimum (if u use for 10 seconds, you still pay for 1min , if u use for 70s u pay for 70s)
 - macOS also is per second but with 24h minimum (if u use for 1h u will still pay for 24h)
- they are billed by
 - compute :
 - the time it is running
 - based on the instance type and the OS
 - storage:
 - the storage it is allocating in ebs ect
 - billed per GB per month
 - traffic:
 - 100GB each month is FREE
 - traffic coming out from the ec2 , first 100gb is free then 0.09 per GB
 - traffic in is totally free
 - internal data: it depends:
 - within the AZ is free

- cross diff AZ (Inter AZ) typically 0.01\$ per GB , only outbound

RDS billing

- the relational databases billing is per 4 things
 - compute: the engine running the database
 - billed per s or per h as long as the instance is running
 - it can be open source (MySQL PostgreSQL etc.) which is cheaper
 - or it can be commercial like Microsoft SQL Server
 - enabling multi AZ for the RDS doubles the compute billing
 - payment offers:
 - on demand pay as u go
 - RI
 - DB Saving plans
 - NO SPOT INSTANCES FOR RDS
 - storage: the ebs storing your db data
 - billed per GB per month , you pay for the storage you allocate per GB
 - enabling multi AZ doubles your cost for storage
 - NB: u keep being billed for storage even if the DB is stopped because the space you are allocating will always take physical space regardless whether the db instance is stopped or not
 - Traffic: data out of your DB
 - data in totally FREE
 - data out is billed per GB , each month you get 100GB for free
 - Backup: AWS takes snapshot on your DB regularly to help you backup and return back
 - totally FREE until your reach the ALLOWANCE
 - the ALLOWANCE is that , if u provision 100GB, AWS will allow u to store 100GB of backups
 - if u exceed that, you will be charged of each GB that is exceeded: 0.095 GB per month
 - snapshots data are stored in S3

VPC billing

- VPCs are almost free,
 - creating VPCs, creating subnets , creating IGW, creating VPC peering , this all is FREE
- what is billed is the components you attach to a VPC, like:
 - NAT GW,

- Interface Endpoints
- Transit GW
- Site to Site VPN
- these all have :
 - hourly cost per hour
 - data processing fee , per GB per month
- also data traffic going in the VPC can be billed:
 - if it is within the same VPC/same AZ, it is FREE
 - cross AZ: 0.01\$ per GB per month each way (IN & OUT)
 - out to the internet 0.09\$ per GB per month
 - internet to AWS 0\$ (INBOUND)
- each public IP assigned to a resource EC2 .. is billed 0.005\$ per hour regardless whether it is active or idle

Lambda billing

- lambda functions are metered by Duration and by Request
 - how many invocations ==> Requests
 - what is the time of execution * allocated Ram ==> Duration (GB-seconds)
- so you pay for two things
 - how many times your code was called
 - and how many GB RAM per second u consumed
- when u create your lambda function you can choose the CPU architecture
 - AMD64 or x86
 - AMD64 is cheaper
- storage:
 - by default you have 512 MB as local storage for FREE
 - if u need more you will PAY
- payment methods
 - on demand
 - compute saving plans
- if u activate **Provisioned Concurrency** u pay more
 - this is used if you want your code to run immediately , because avg lambda takes time to warm up , ULTA LOW LATENCY
 - so your code will 24/7 be warm, just like a subtle server that is listening 24/7